Software Design Specifications

for

Restaurant Reservation System

Prepared by:
Nareen Reddy - SE23UCSE120
Charith Leburu - SE23UARI074
Aryan Goud - SE32UARI038
Pranav Akkina - SE23UCSE144
Shanvi Rishi -SE23UARI136
Sejal - SE23UCSE209
Swetha Burra - SE23UCSE170
Sreethan Reddy - SE23UCSE138

Document Information

Title: Restaurant Reservation System	
Project Manager:Nareen	**Document Version No:1**
	Document Version Date:6/04/2026
Prepared By:G17	**Preparation Date:**

Table of Contents

1 Introduction

1.1 Purpose

This document provides a detailed design specification for the Restaurant Reservation System. It describes how the system will be structured and implemented based on the requirements defined in the SRS.

The document is intended for developers, testers, and project evaluators. It helps developers understand system architecture, module interactions, and data handling, while also serving as a reference during implementation and testing.

1.2 Scope

This document applies to the design and implementation of the Restaurant Reservation System. It covers system architecture, module design, database structure, interfaces, and data flow.

The system enables users to search restaurants, view details, make reservations, and perform payments.

1.3 Definitions, Acronyms, and Abbreviations

API – Application Programming Interface
DB – Database
UI – User Interface
UX – User Experience
SRS – Software Requirements Specification
SDS – Software Design Specification

1.4 References

SRS Document – Restaurant Reservation System
IEEE SRS Guidelines
React Documentation – https://react.dev
Node.js Documentation – https://nodejs.org
PostgreSQL Documentation – https://www.postgresql.org/docs

2 Use Case View

[Based on the requirements document, this section lists use cases or scenarios from the use-case model if they represent some significant, central functionality of the final system, or if they have a large design coverage - they exercise many design elements, or if they stress or illustrate a specific, delicate point of the design. Provide a use case diagram for use cases that pertains to the software design]

2.1 Use Case

The system supports the following major use cases:

- Login
- Make Reservation
- Modify Reservation
- Cancel Reservation
- Make Payment

Example: Make Reservation

1. User selects a restaurant
2. User enters date, time, guests
3. System checks availability
4. System confirms reservation
5. Confirmation is sent to user

3 Design Overview

3.1 Design Goals and Constraints

1. Design Goals:

- Provide a simple and responsive UI
- Ensure fast reservation processing
- Prevent overbooking
- Maintain secure user authentication

2. Constraints:

- Must use React, FastAPI, PostgreSQL
- Must run on web browsers
- Limited development timeline
- Internet dependency

3.2 Design Assumptions

- Users have internet access
- Restaurants provide accurate availability data
- Payment gateway APIs are reliable
- Users access system via modern browsers

3.3 Significant Design Packages

The system is divided into three main layers:

1. Presentation Layer (Frontend)

- Built using React
- Handles UI and user interactions

2. Application Layer (Backend)

- Built using FastAPI(python)
- Handles business logic and APIs

3. Data Layer (Database)

- PostgreSQL database
- Stores users, restaurants, reservations

3.4 Dependent External Interfaces

The table below lists the public interfaces this design requires from other modules or applications.

External Application and Interface Name	Module Using the Interface	Functionality/ Description
Payment Gateway	Payment Module	Processes payments securely
Web Browser	Frontend	Displays the UI
PostgreSQL	Backend	Stores system data

3.5 Implemented Application External Interfaces (and SOA web services)

The table below lists the implementation of public interfaces this design makes available for other applications.

Interface Name	Module	Description
User API	backend	Handles login and reservation
Reservation API	backend	Manages booking operations
Payment API	backend	Integrates payment gateway

4 Logical View

4.1 Design Model

The system is composed of the following main classes:

Key Classes:

User

- Attributes: user_id, user_name, email, phone, password_hash
- Responsibilities: authentication, reservation management

Restaurant

- Attributes: restaurant_id, name, location, cuisine
- Responsibilities: provide restaurant details, ratings

Table

- Attributes: table_id, restaurant_id, capacity
- Responsibilities: manage seating capacity

Reservation

- Attributes: reservation_id, date, time, guests, status
- Responsibilities: booking, updating, cancellation

Review

- Attributes: review_id, rating, comment
- Responsibilities: store user feedback

Relationships:

- User → Reservation (1:M)
- Restaurant → Reservation (1:M)

4.2 Use Case Realization

Make Reservation Flow:

1. *User → Frontend*
2. *Frontend → Backend API*
3. *Backend → Database (check availability)*
4. *Backend → Database (store reservation)*
5. *Backend → Frontend (confirmation)*

5 Data View

5.1 Domain Model

The system is built around five main domain entities:

- **Users** – Represents customers using the platform
- **Restaurants** – Stores restaurant details
- **Restaurant Tables** – Represents individual tables in each restaurant
- **Reservations** – Stores booking details made by users
- **Reviews** – Stores user feedback and ratings for restaurants

Relationships:

- A **User** can make multiple **Reservations** (1:M)
- A **Restaurant** can have multiple **Tables** (1:M)
- A **Restaurant** can have multiple **Reservations** (1:M)
- A **Reservation** is linked to a specific **Table**
- A **User** can give multiple **Reviews** (1:M)
  A **Restaurant** can receive multiple **Reviews** (1:M)

5.2 Data Model (persistent data view)

Tables:

Users

- user_id (PK)
- name
- email
- password
- password_hash
- created_at

Restaurants

- restaurant_id (PK)
- name
- location
- cuisine
- opening_time
- closing_time
- avg_rating
- created_at

Reservations

- reservation_id(PK)
- user_id(int4 (FK))
- table_id(int4 (FK))
- restaurant_id(int4 (FK))
- reservation_date(date)
- reservation_time(time)
- guests(int4)
- status(varchar)
- created_at(timestamp)

Reviews Table

- **review_id**: int4 (PK)
- **user_id**: int4 (FK)
- **restaurant_id**: int4 (FK)
- **rating**: int4
- **comment**: text
- **created_at**: timestamp

5.2.1 Data Dictionary

Field	Description
user_id	Unique identifier for users
restaurant_id	Unique identifier for restaurants
table_id	Unique identifier for tables
reservation_id	Unique booking ID
review_id	Unique review ID
status	Indicates reservation state (Confirmed/Cancelled/Pending)
avg_rating	Calculated average of all restaurant reviews

6 Exception Handling

- Invalid login → Show error message
- No table available → Prompt user to change time
- Payment failure → Retry option
- Server error → Display system message

All exceptions are logged for debugging.

7 Configurable Parameters

This table describes the simple configurable parameters (name / value pairs).

Configuration Parameter Name	Definition and Usage	Dynamic?
Max Tables	Maximum tables per restaurant	Yes
Timeout	Session timeout duration	Yes
Payment Retry Limit	Maximum retries allowed	No

8 Quality of Service

8.1 Availability

- System available 24/7
- Minimal downtime

8.2 Security and Authorization

- User authentication required
- Password encryption
- Secure APIs (HTTPS)

8.3 Load and Performance Implications

- Supports multiple concurrent users
- Fast query execution (<2 sec)

8.4 Monitoring and Control

- Logs system activity
- Tracks errors and failures
- Admin dashboard for monitoring